

클래스의 완전한 작동 구현

```
class Car:
    color = ""
    speed = 0

    def upSpeed(self, value) :
        self.speed += value

    def downSpeed(self, value) :
        self.speed -= value

myCar1 = Car()
myCar1.color = "빨강"
myCar1.speed = 0

myCar2 = Car()
myCar2.color = "파랑"
myCar2.speed = 0

myCar3 = Car()
myCar3.color = "노랑"
myCar3.speed = 0

myCar1.upSpeed(30)
print(f"자동차 1의 색상은 {myCar1.color}이며, 현재 속도는 {myCar1.speed}입니다")

myCar2.upSpeed(60)
print(f"자동차 2의 색상은 {myCar2.color}이며, 현재 속도는 {myCar2.speed}입니다")

myCar3.upSpeed(0)
print(f"자동차 3의 색상은 {myCar3.color}이며, 현재 속도는 {myCar3.speed}입니다")

자동차 1의 색상은 빨강이며, 현재 속도는 30 입니다
자동차 2의 색상은 파랑이며, 현재 속도는 60 입니다
자동차 3의 색상은 노랑이며, 현재 속도는 0 입니다
```

기본 생성자

매개변수가 self 만 있는 생성자

```
class Car :
    color = ""
    speed = 0

    def __init__(self):
        self.color = "빨강"
        self.speed = 0
```

```
def upSpeed(self, value) :
    self.speed += value

def downSpeed(self, value) :
    self.speed -= value
```

매개변수가 있는 생성자

```
class Car :
    color = ""
    speed = 0

    def __init__(self, value1, value2):
        self.color = value1
        self.speed = value2

myCar1 = Car("빨강", 30)
myCar2 = Car("파랑", 60)
```

기존 프로그램의 완성버전

```
class Car :
    name = ""
    speed = 0

    def __init__(self, name, speed):
        self.name = name
        self.speed = speed

    def getName(self) :
        return self.name

    def getSpeed(self) :
        return self.speed

car1, car2 = None, None

car1 = Car("아우디", 0)
car2 = Car("벤츠", 30)

print(f"{car1.getName()}의 현재 속도는 {car1.getSpeed()}입니다.")
print(f"{car2.getName()}의 현재 속도는 {car2.getSpeed()}입니다.")

아우디의 현재 속도는 0 입니다.
벤츠의 현재 속도는 30 입니다.
```

인스턴스 변수

예: Car 클래스 2 개의 필드

```
class Car :
    color = ""
    speed = 0
```

자동차 생산 대수 확인 코드

```
class Car :
    color = "" # 인스턴스 변수
    speed = 0 # 인스턴스 변수
    count = 0 # 클래스 변수

    def __init__(self):
        self.speed = 0
        Car.count += 1

myCar1, myCar2 = None, None

myCar1 = Car()
myCar1.speed = 30
print(f"자동차 1의 현재 속도는 {myCar1.speed}km, 생산된 자동차는 총 {Car.count}대
입니다.")

myCar2 = Car()
myCar2.speed = 60
print(f"자동차 2의 현재 속도는 {myCar2.speed}km, 생산된 자동차는 총 {myCar2.count}
대 입니다.")

자동차 1의 현재 속도는 30km, 생산된 자동차는 총 1 대 입니다.
자동차 2의 현재 속도는 60km, 생산된 자동차는 총 2 대 입니다.
```

메서드 오버라이딩 구현

```
class Car :
    speed = 0
    def upSpeed(self, value) :
        self.speed += value

        print(f"현재 속도 (슈퍼 클래스) : {self.speed}")

class Sedan(Car) :
    def upSpeed(self, value):
        self.speed += value

        if self.speed > 150 :
            self.speed = 150
            print(f"현재 속도 (서브 클래스) : {self.speed}")

class Truck(Car) :
    pass
```

```

sedan1, truck1 = None, None

truck1 = Truck()
sedan1 = Sedan()

print("트럭 --> ", end="")
truck1.upSpeed(200)

print("승용차 --> ", end="")
sedan1.upSpeed(200)

트럭 --> 현재 속도 (슈퍼 클래스) : 200
승용차 --> 현재 속도 (서브 클래스) : 150

```

super() 함수

서브 클래스에서 메서드 오버라이딩을 할 때, 슈퍼 클래스의 메서드나 속성을 사용해야 하는 경우가 있다. 이럴 때는 super() 메서드를 사용하면 된다

```

class Car :
    value = '슈퍼 값'
    def carMethod(self) :
        print("슈퍼 클래스 메서드 실행~~~")

class Sedan(Car) :
    value = '서브 값'

    def carMethod(self):
        super().carMethod() # Car 클래스의 carMethod 가 실행됨
        print('서브 클래스 메서드 실행~~')
        print(super().value)

sedan1 = Sedan()
sedan1.carMethod()

슈퍼 클래스 메서드 실행~~~
서브 클래스 메서드 실행~~
슈퍼 값

```

클래스의 특별한 메서드

`__del__()`

- 소멸자(Destructor)
- 생성자와 반대로 인스턴스가 삭제될 때 자동으로 호출되는 메서드

`__repr__()`

- 인스턴스를 `print()` 문으로 출력할 때 실행되는 메서드
-

`__add__()`

- 인스턴스 사이에 덧셈 연산이 발생할 때 실행되는 메서드
 - 인스턴스끼리 덧셈 작업을 수행할 수 있음
-

비교 메서드

- `__lt__()`
- `__le__()`
- `__gt__()`
- `__ge__()`
- `__eq__()`
- `__ne__()`

인스턴스 사이에서 비교 연산자(<, <=, >, >=, ==, !=)를 사용할 때 호출되는 메서드

```
class Line :
    length = 0
    def __init__(self, length):
        self.length = length

        print(f"{self.length} 길이의 선이 생성되었습니다.")

    def __del__(self) :
        print(f"{self.length} 길이의 선이 삭제되었습니다.")

    def __repr__(self):
        return f'선의 길이 : {self.length}'

    def __add__(self, other):
        return self.length + other.length

    def __lt__(self, other):
        return self.length < other.length

    def __eq__(self, other):
        return self.length == other.length

myLine1 = Line(100)
myLine2 = Line(200)

print(myLine1)

print(f"두 선의 길이 합 : {myLine1 + myLine2}")
```

```

if myLine1 < myLine2 :
    print("선분 2가 더 기네요.")

elif myLine1 == myLine2 :
    print('두 선분이 같네요.')

else :
    print('모르겠네요')

del(myLine1)

100 길이의 선이 생성되었습니다.
200 길이의 선이 생성되었습니다.
선의 길이 : 100
두 선의 길이 합 : 300
선분 2가 더 기네요.
100 길이의 선이 삭제되었습니다.

```

추상 메서드

서브 클래스에서 메서드를 오버라이딩: 슈퍼 클래스에서는 빈 껍질의 메서드만 만들어 놓고 내용은 pass로 채움

```

class SuperClass:
    def method(self) :
        pass

class SubClass1(SuperClass) :
    def method(self):
        print('SubClass1에서 method()를 오버라이딩함')

class SubClass2(SuperClass) :
    pass

sub1 = SubClass1()
sub2 = SubClass2()

sub1.method()
sub2.method()

SubClass1에서 method()를 오버라이딩함

# 자동차 세 대가 경주하는 코드
import time

class RacingCar :

```

```

carName = ''
def __init__(self, name):
    self.carName = name

def runCar(self) :
    for _ in range(0, 3) :
        carStr = self.carName + '~~달립니다.\n'
        print(carStr, end='')
        time.sleep(0.1)

car1 = RacingCar('@자동차 1')
car2 = RacingCar('#자동차 2')
car3 = RacingCar('$자동차 3')

car1.runCar()
car2.runCar()
car3.runCar()

```

```

@자동차 1~~달립니다.
@자동차 1~~달립니다.
@자동차 1~~달립니다.
#자동차 2~~달립니다.
#자동차 2~~달립니다.
#자동차 2~~달립니다.
$자동차 3~~달립니다.
$자동차 3~~달립니다.
$자동차 3~~달립니다.

```

자동차 세 대를 동시에 출발

```

import threading
import time

class RacingCar :
    carName = ''
    def __init__(self, name):
        self.carName = name

    def runCar(self) :
        for _ in range(0, 3) :
            carStr = self.carName + '~~달립니다.\n'
            print(carStr, end='')
            time.sleep(0.1)

car1 = RacingCar('@자동차 1')
car2 = RacingCar('#자동차 2')
car3 = RacingCar('$자동차 3')

th1 = threading.Thread(target=car1.runCar)
th2 = threading.Thread(target=car2.runCar)
th3 = threading.Thread(target=car3.runCar)

```

```
th1.start()  
th2.start()  
th3.start()
```

```
@자동차 1~~달립니다.  
#자동차 2~~달립니다.  
$자동차 3~~달립니다.
```

```
@자동차 1~~달립니다.  
$자동차 3~~달립니다.  
#자동차 2~~달립니다.  
#자동차 2~~달립니다.  
$자동차 3~~달립니다.  
@자동차 1~~달립니다.
```